

## 1. O que é autenticação?

Autenticação responde à pergunta:

"Quem é você?"

Exemplo:

- Usuário: admin
- Senha: 123456

Se as credenciais estiverem corretas, o sistema permite o acesso.

---

## 2. O que é autorização?

Autorização responde à pergunta:

"O que você pode fazer?"

Exemplo:

| Usuário | Papel | Permissão       |
|---------|-------|-----------------|
| admin   | ADMIN | CRUD completo   |
| joao    | USER  | Apenas consulta |

---

### Exemplo 1 - Autenticação Básica

#### Dependência Maven

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-security</artifactId>  
</dependency>
```

Ao adicionar essa dependência, o Spring já protege todos os endpoints.

---

#### Controller

```
@RestController  
@RequestMapping("/produtos")  
public class ProdutoController {  
  
    @GetMapping
```

```
public List<String> listar() {  
    return List.of(  
        "Notebook",  
        "Mouse",  
        "Teclado"  
    );  
}  
}
```

---

## Configuração de Usuários

@Configuration

```
public class SecurityConfig {
```

@Bean

```
public UserDetailsService userDetailsService() {
```

```
    UserDetails admin =
```

```
        User.builder()
```

```
            .username("admin")
```

```
            .password(passwordEncoder().encode("123"))
```

```
            .roles("ADMIN")
```

```
            .build();
```

```
    UserDetails user =
```

```
        User.builder()
```

```
            .username("joao")
```

```
            .password(passwordEncoder().encode("123"))
```

```
            .roles("USER")
```

```
            .build();
```

```
    return new InMemoryUserDetailsManager(  
        admin,
```

```
        user
```

```
    );
```

```
}
```

@Bean

```
public PasswordEncoder passwordEncoder() {
```

```
    return new BCryptPasswordEncoder();
```

```
}  
}
```

---

## **Liberando o acesso via HTTP Basic**

```
@Configuration  
@EnableWebSecurity  
public class SecurityConfig {  
  
    @Bean  
    public SecurityFilterChain filterChain(  
        HttpSecurity http) throws Exception {  
  
        return http  
            .csrf(csrf -> csrf.disable())  
            .authorizeHttpRequests(auth -> auth  
                .anyRequest()  
                .authenticated()  
            )  
            .httpBasic(Customizer.withDefaults())  
            .build();  
    }  
}
```

---

## **Testando**

### **Sem usuário**

GET /produtos

Resposta:

401 Unauthorized

---

### **Com usuário**

GET /produtos

Authorization: Basic YWRtaW46MTIz

ou usando Postman:

Username: admin

Password: 123

Resposta:

```
[  
  "Notebook",  
  "Mouse",  
  "Teclado"  
]
```

---

### Como funciona internamente?

Quando a requisição chega:

Cliente

```
|  
| usuário + senha  
v
```

Spring Security Filter

```
|  
| valida credenciais  
v
```

UserDetailsService

```
|  
| busca usuário  
v
```

PasswordEncoder

```
|  
| compara senha  
v
```

Controller

---

### Exemplo 2 - Usuários no Banco de Dados

#### Entidade

@Entity

@Table(name = "usuarios")

public class Usuario {

@Id

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;

private String username;

private String password;

private String role;

// getters e setters
}
```

---

### **Repository**

```
@Repository
public interface UsuarioRepository
    extends JpaRepository<Usuario, Long> {

    Optional<Usuario> findByUsername(
        String username);
}
```

---

### **UserDetailsService**

```
@Service
public class UsuarioDetailsService
    implements UserDetailsService {

    private final UsuarioRepository repository;

    public UsuarioDetailsService(
        UsuarioRepository repository) {
        this.repository = repository;
    }

    @Override
    public UserDetails loadUserByUsername(
        String username)
        throws UsernameNotFoundException {
```

```
        Usuario usuario =
            repository.findByUsername(username)
                .orElseThrow(() ->
                    new UsernameNotFoundException(
                        "Usuário não encontrado"));

        return User.builder()
            .username(usuario.getUsername())
            .password(usuario.getPassword())
            .roles(usuario.getRole())
            .build();
    }
}
```

---

### **Cadastro de Usuário**

```
@PostMapping("/usuarios")
public Usuario cadastrar(
    @RequestBody Usuario usuario) {

    usuario.setPassword(
        passwordEncoder.encode(
            usuario.getPassword()));

    return repository.save(usuario);
}
```

---

### **Autorização por Perfil**

#### **Apenas ADMIN pode cadastrar**

```
@PostMapping
@PreAuthorize("hasRole('ADMIN')")
public Produto salvar(
    @RequestBody Produto produto) {

    return service.salvar(produto);
}
```

---

#### **Apenas usuários autenticados podem listar**

```

@GetMapping
@PreAuthorize("isAuthenticated()")
public List<Produto> listar() {
    return service.listar();
}

```

Ative o método:

```

@Configuration
@EnableMethodSecurity
public class SecurityConfig {
}

```

---

## Exemplo Prático de Aula

Imagine uma API de produtos.

| Endpoint              | Acesso        |
|-----------------------|---------------|
| GET /produtos         | USER ou ADMIN |
| GET /produtos/{id}    | USER ou ADMIN |
| POST /produtos        | ADMIN         |
| PUT /produtos/{id}    | ADMIN         |
| DELETE /produtos/{id} | ADMIN         |

Configuração:

```

@Bean
public SecurityFilterChain filterChain(
    HttpSecurity http) throws Exception {

    return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers(HttpMethod.GET,
                "/produtos/**")
            .hasAnyRole("USER", "ADMIN")

            .requestMatchers(HttpMethod.POST,
                "/produtos/**")
            .hasRole("ADMIN")
        )
    ;
}

```

```

        .requestMatchers(HttpMethod.PUT,
            "/produtos/**")
        .hasRole("ADMIN")

        .requestMatchers(HttpMethod.DELETE,
            "/produtos/**")
        .hasRole("ADMIN")

        .anyRequest()
        .authenticated()
    )
    .httpBasic(
        Customizer.withDefaults())
    .build();
}

```

---

## Limitações do Basic Authentication

Em aplicações modernas, normalmente usamos **JWT (JSON Web Token)** porque:

- Não envia usuário e senha a cada requisição.
- É stateless.
- Funciona melhor com microsserviços.
- É o padrão para APIs REST.

Fluxo JWT:

Login

|

| usuário + senha

v

API

|

| gera Token JWT

v

Cliente

Token:

eyJhbGciOiJIUzI1Ni...



Cliente envia:

Authorization: Bearer eyJhbGciOiJIUzI1Ni...